

VENOM – capa de inteligencia operativa

David Fernando Mora Sabogal

Daniel Estiven Gonzalez torres

Gestor del conocimiento

German Ortiz Diaz

Universidad de Cundinamarca

Ingeniería de sistema de computación

2026

Contenido

Introducción	3
Descripción	4
Justificación.....	6
Alcance del Proyecto.....	7
Arquitectura del Sistema.....	9
Metodología de los 12 Factores	15
Metodología	17
Backlog	19
Projectlibre	29
Requerimientos	30
Dependencias	32
Configuración tecnológica	33
Riesgos	33
Base de datos.....	34
Diagrama de flujo.....	38
Casos de uso.....	40
Historia de usuario	40
Diagrama uml.....	41
Pruebas	42

Propuestas de mejora.....	63
Conclusiones	65
Referencias.....	66

Introducción

En el ámbito empresarial moderno las empresas requieren de cada vez mas datos debido a la transformación digital, al ser mucha información muchas veces se tienen una mala gestión en los mismo llegando muchas veces a aislarlo, convirtiendo el proceso de consulta de datos en uno tedioso para una empresa la cual requiera el manejo de múltiples datos para su buen funcionamiento, ya que si estos datos aislados llegan a perderse o se llegan a ver afectados un microempresa prestadora de servicio puede generar perdidas, como clientes que pierden la confianza en el procesamiento y manejo de datos de la empresa perdiendo el interés en el mismo servicio, como solución también muchas empresas buscan dedicar una persona encargada de manejar toda esta información, en organizarla y distribuirla cuando se requiera, una alternativa que necesita de una mano de obra adicional que puede llegar a ser costosa en diversas empresas que no cuentan con un presupuesto para incluir esta mano de obra en su empresa.

Descripción

Problema identificado

En el entorno empresarial actual, la transformación digital ha generado un volumen masivo de datos. Sin embargo, esta abundancia ha derivado en una fragmentación crítica; la información se encuentra aislada en diferentes silos, lo que convierte los procesos de consulta en tareas tediosas y propensas al error. Para las microempresas, esto se traduce en una pérdida de agilidad operativa y una disminución en la confianza del cliente. Actualmente, la solución convencional es asignar personal dedicado exclusivamente a la gestión de datos, lo que representa un costo operativo elevado que muchas organizaciones no pueden asumir.

Objetivo general

Desarrollar una Capa de Inteligencia Operativa (OIL - *Operational Intelligence Layer*) denominada VENOM, que centralice, organice y distribuya el conocimiento empresarial mediante el uso de arquitecturas *multi-tenant* y tecnologías de Recuperación Aumentada por Generación (RAG), permitiendo consultas eficientes y automatizadas.

Objetivos específicos

- Implementar un sistema de lecturas de documento (RAG) para que la IA responda basándose en manualidades y bases de datos reales.
- Construir una extensión de navegador fluida que permita acceder a la IA sin salir de las herramientas de trabajo habituales.

- Programar una lógica de roles que diferencie las respuestas para empleados y para clientes
- Conectar la IA con fuentes de datos externas (APIs/SQL) para que pueda consultar estados de servicios o inventarios en tiempo real

Usuarios del sistema

Rol	Descripción
Administrador	Crea y monitorea tenants globales, gestiona la base de conocimiento de su organización, carga documentos y configura parámetros de IA por empresa
Usuario	Usuario que interactúa con la IA (Venom) para realizar consultas rápidas al internet y sobre los documentos vectorizados

Justificación

La implementación de una capa inteligente operativa (OIL) se justifica por la creciente complejidad a la hora de manejar un flujo de información dentro de organizaciones modernas, de aquí surge una nueva necesidad donde en la actualidad existen alternativas como softwares tradicionales, este proyecto a diferencia de los anteriormente mencionados software modernos no busca reemplazar los sistemas existentes, como los CMRs, las bases de datos o procedimientos manuales en las empresas, sino ser una herramienta que busque potenciar todos estos procesos anteriormente mencionados por medio de un interfaz inteligente y ubicua.

La técnica de este proyecto radica en su naturaleza "simbiótica". Al proponerse como una extensión de navegador, la Capa de Inteligencia Operativa se integra directamente en el espacio de trabajo del usuario. Esto permite una adopción inmediata sin necesidad de costosas migraciones de datos o cambios en la infraestructura actual de la empresa, garantizando un retorno de inversión (ROI) más rápido.

Alcance del Proyecto

El desarrollo de **VENOM** comprende el diseño e implementación de una solución integral estructurada en los siguientes niveles técnicos:

1. **Interfaz de Usuario (Frontend Funcional):** Se desarrollará una extensión de navegador utilizando el framework **React** y **TypeScript**, diseñada para proporcionar una interacción fluida y no invasiva. Este frontend permitirá al usuario realizar consultas, visualizar respuestas contextuales y gestionar el cambio de roles (interno/externo) de manera intuitiva.
2. **Capa de Lógica de Negocio (Backend):** Se construirá un núcleo operativo basado en **Python** con el framework **FastAPI**, encargado de orquestar los procesos críticos del sistema. Este componente gestionará la recuperación de información mediante técnicas de **RAG**, la validación de permisos de usuario y la canalización de datos hacia los modelos de lenguaje de gran escala (LLM).
3. **Sistema de Persistencia de Información:** El proyecto integrará una arquitectura de datos híbrida que garantiza la continuidad operativa. Se utilizará una **base de datos vectorial** para el almacenamiento y búsqueda semántica de documentos corporativos, junto con una **base de datos relacional** para persistir configuraciones, historiales de chat y metadatos de los clientes.
4. **Protocolos de Comunicación Eficiente:** Se establecerá una conectividad técnica robusta entre el frontend y el backend a través de una **API RESTful**. La comunicación se realizará mediante el intercambio de objetos **JSON**, optimizando el tráfico de red.

y asegurando una latencia mínima para cumplir con los estándares de respuesta en tiempo real requeridos en entornos de soporte.

5. **Estándares y Buenas Prácticas de Ingeniería:** El ciclo de vida del desarrollo se regirá por principios de **arquitectura limpia (Clean Architecture)** y patrones de diseño **SOLID**. Se implementará un código modular, documentado y escalable, aplicando protocolos de seguridad para la protección de datos sensibles y asegurando que la solución sea fácilmente mantenible y adaptable a diversos dominios empresariales.

Arquitectura del Sistema

La arquitectura de VENOM se define como un ecosistema desacoplado donde la interfaz de usuario y el motor de inteligencia operativa se comunican mediante protocolos asíncronos, garantizando escalabilidad y mantenibilidad.

6.1 Arquitectura General

El sistema se organiza en tres capas tecnológicas claramente diferenciadas:

- Capa de Cliente (Frontend): Desarrollada en React y TypeScript, empaquetada como una extensión de navegador.
- Capa de Servicios (Backend): Un núcleo de procesamiento construido en Python (FastAPI) para la lógica de IA y TypeScript (Node.js) si se requiere un orquestador de API adicional (o integrado directamente en el backend de Python).
- Capa de Persistencia (Data): Un motor híbrido que utiliza SQL (PostgreSQL) para datos relacionales y una base de datos vectorial para el motor RAG.

Relación de Componentes:

1. La extensión (React/TS) envía peticiones autenticadas vía JSON.
2. El backend (Python/FastAPI) procesa la lógica de negocio y realiza la búsqueda semántica.
3. La base de datos SQL valida el usuario y el rol, mientras que la base Vectorial provee el contexto técnico.

6.2 Frontend (Interfaz y Experiencia)

El frontend no es una página web convencional, sino un componente inyectado en el flujo de trabajo del navegador.

- Tecnologías: React 18 para la UI y TypeScript para asegurar la integridad de los tipos de datos en la comunicación con la API.
 - Navegación: Gestión de estados mediante *Hooks* (`useState`, `useEffect`) para alternar entre el chat de soporte, el panel de configuración de la empresa y el historial de consultas.
 - Consumo de Servicios: Implementación de servicios mediante Axios con interceptores para manejar tokens de seguridad y respuestas de la Capa de Inteligencia Operativa.
 - Validaciones: Validación estática con TypeScript y validación dinámica de formularios para asegurar que la entrada del usuario sea limpia antes de enviarla al backend.
-

6.3 Backend (Cerebro Operativo)

El backend es donde reside la potencia de VENOM, utilizando Python por su supremacía en el área de la Inteligencia Artificial.

- Exposición de API: Uso de FastAPI (Python). Este framework permite definir endpoints asíncronos que manejan la comunicación con los modelos de lenguaje (LLM) sin bloquear el sistema.
- Lógica de Negocio (TypeScript/Python): * Uso de TypeScript en los scripts de la extensión para manejar la lógica de la interfaz.
 - Uso de Python para el motor de Simbiosis de Datos, que fragmenta manuales (PDF/JSON) y los convierte en conocimiento consultable.
- Gestión de Datos (SQL): Implementación de una base de datos SQL (PostgreSQL) para el almacenamiento de:

- Perfiles de usuario y niveles de acceso (Roles).
 - Logs de auditoría (quién preguntó qué y cuándo).
 - Configuraciones específicas de la microempresa cliente.
- Validaciones del Servidor: Uso de Pydantic en el backend para validar que los objetos JSON recibidos cumplen con el contrato de la API, evitando errores de ejecución.

6.4 Capa de Persistencia e IA

Para que VENOM sea una verdadera "Capa de Inteligencia", la gestión de datos es doble:

1. Motor de Búsqueda Semántica: Los documentos de la empresa se procesan con Python y se almacenan en una base de datos de vectores. Esto permite que la IA "encuentre" la respuesta correcta en un manual de 200 páginas en milisegundos.
2. Repositorio Relacional (SQL): Se utiliza SQL para asegurar que la información operativa (como estados de cuenta o tickets de soporte) sea exacta y coherente, aplicando reglas de integridad referencial.

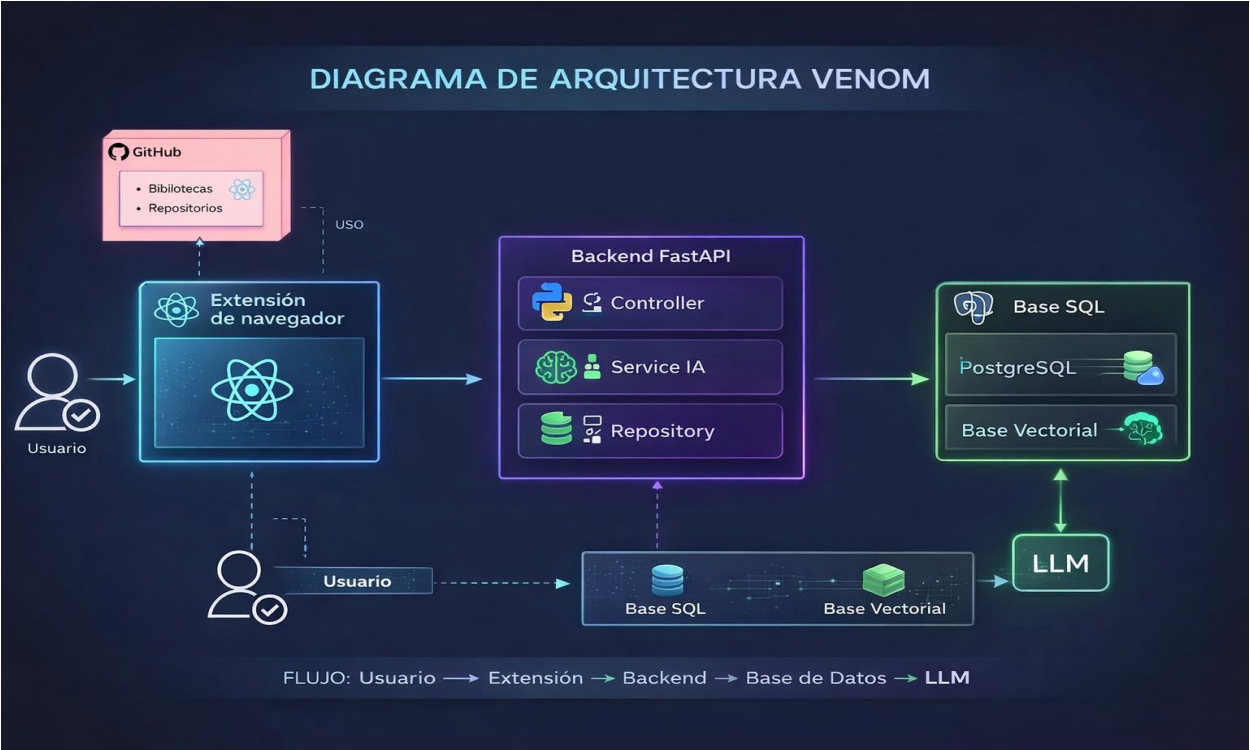
Tabla de arquitectura

El sistema VENOM utiliza una arquitectura distribuida basada en microservicios desacoplados, optimizada para la nube y entornos de alta disponibilidad.

Capa	Tecnología	Responsabilidad
Frontend	React	Interfaz de usuario responsiva, gestión de estado y comunicación con la API
Backend	Phyton (FastAPI)	Lógica de negocio,

		orquestración de servicios de IA y gestión de autenticación.
Base de datos	postgreSQL	Gestión de datos de usuarios, empresas (Multi-tenancy) y logs.
Infraestructura	Docker compose	Contenerización del proyecto para su despliegue efectivo en diferentes entornos

Diagrama de arquitectura



Justificación de arquitectura

Frontend

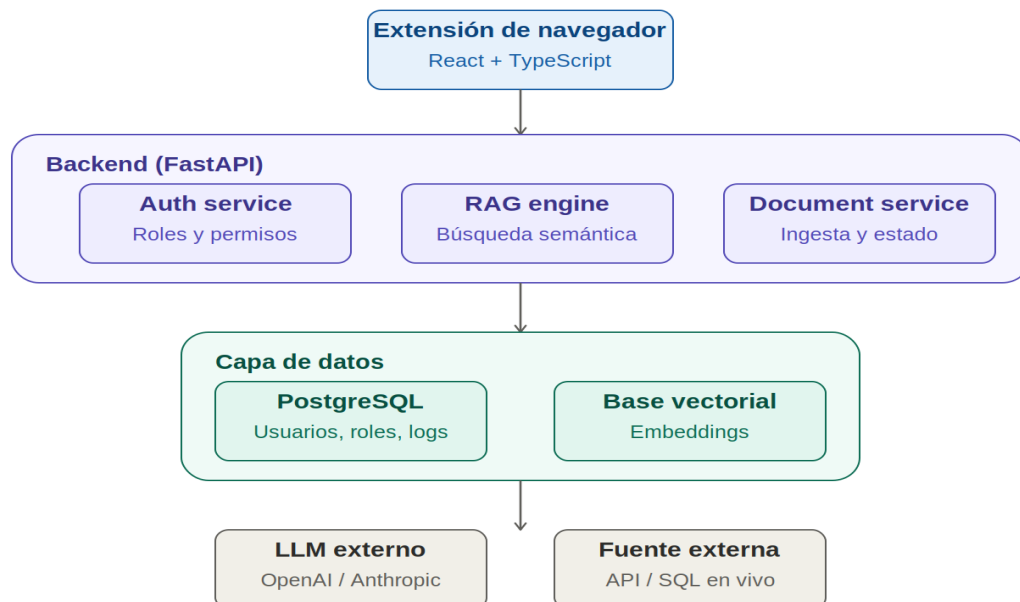
Se implementa una arquitectura modular que separa componentes y servicios. Esto permite que el sistema sea escalable y fácil de mantener a un largo plazo y siguiendo las buenas practicas para que no sea un problema el pasar el proyecto a algún compañero

Backend

Se utiliza FastAPI por su capacidad asíncrona, lo cual es vital al realizar llamadas a modelos de IA que pueden tener latencia. La estructura sigue principios de responsabilidad única para asegurar que cada endpoint realice una tarea específica sin efectos secundarios imprevistos.

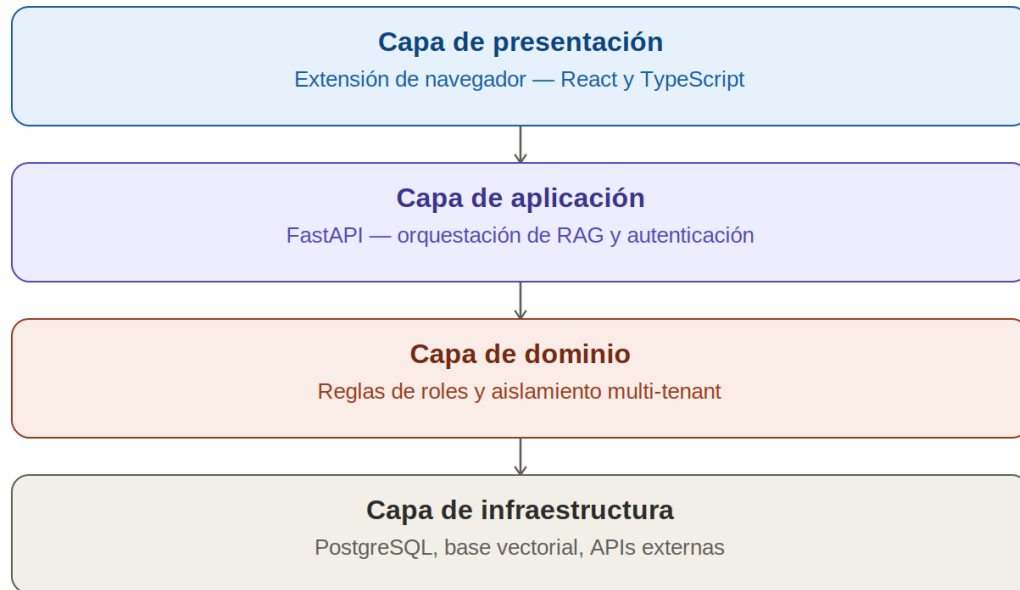
Diagrama de componentes

El siguiente diagrama detalla los componentes internos que conforman el backend de VENOM (Auth Service, RAG Engine y Document Service), y cómo estos se relacionan con la extensión de navegador, la capa de datos y los servicios externos de inteligencia artificial.



Arquitectura de software

VENOM sigue un enfoque de arquitectura limpia (Clean Architecture) organizado en cuatro capas, donde cada capa únicamente depende de la inmediatamente inferior, facilitando el mantenimiento y la sustitución de tecnologías sin afectar la lógica de negocio.



Metodología de los 12 Factores

En el desarrollo de Project Venom, hemos adoptado la metodología de los 12 Factores para garantizar que el sistema sea escalable, portátil y profesional. A continuación, se detallan los factores que hemos implementado rigurosamente hasta la fase actual:

I. Codebase (Código Base)

Explicación: Mantenemos un único repositorio de código compartido para todos los despliegues.

Aplicación en Venom: Utilizamos Git para gestionar el historial de versiones. Aunque el proyecto tiene un Frontend (React) y un Backend (FastAPI), cada uno se gestiona como una entidad única, permitiendo que cualquier desarrollador pueda replicar el entorno exacto de trabajo sin discrepancias entre versiones.

II. Dependencies (Dependencias)

Explicación: La aplicación nunca debe dar por hecho la existencia de herramientas en el sistema global; todas las dependencias se declaran explícitamente.

Aplicación en Venom: Utilizamos entornos virtuales (.venv) y archivos de requisitos (requirements.txt/ package.json). Esto garantiza que, al mover el proyecto de un computador de mesa a un portátil, el sistema sepa exactamente qué versiones de Groq, FastAPI o React necesita para "sobrevivir".

III. Config (Configuración)

Explicación: Todo lo que pueda variar entre entornos (desarrollo, prueba, producción) se guarda en variables de entorno, nunca dentro del código.

Aplicación en Venom: Implementamos un archivo config.py que lee desde un .env. Las credenciales de la base de datos PostgreSQL y la API Key de Groq están aisladas del código

fuelle. Esto permite que el "simbionte" se conecte a diferentes bases de datos sin necesidad de modificar ni una sola línea de lógica programática.

IV. Backing Services (Servicios de Respaldo)

Explicación: Los servicios como bases de datos o sistemas de mensajería se tratan como recursos adjuntos.

Aplicación en Venom: Para Venom, PostgreSQL (datos relacionales) y ChromaDB (datos vectoriales) son servicios externos. Nuestra arquitectura permite que, si mañana decidimos mover la base de datos a la nube (como Neon.tech), solo tengamos que cambiar la URL de conexión, tratando al servicio de datos como un recurso "enchufable".

VI. Stateless Processes (Procesos sin Estado)

Explicación: La aplicación se ejecuta como uno o más procesos que no guardan datos en su propia memoria local entre peticiones.

Aplicación en Venom: El backend de FastAPI es totalmente stateless. Toda la información persistente se guarda en Postgres o ChromaDB. Si el servidor se reinicia, no se pierde información, ya que el estado vive en los servicios de respaldo, no en el proceso de ejecución.

XI. Logs (Historial de Eventos)

Explicación: Los logs deben tratarse como un flujo de eventos continuos, permitiendo observar el comportamiento del sistema en tiempo real.

Aplicación en Venom: Hemos integrado Loguru para gestionar un flujo de eventos detallado. No solo imprimimos errores; registramos el tiempo de respuesta de la IA, el éxito de las consultas vectoriales y los intentos de acceso. Esto convierte a los logs en una herramienta de diagnóstico proactivo, fundamental para entender cómo el simbionte interactúa con los datos de cada empresa

Metodología

Análisis

Para poder definir la estrategia de desarrollo adecuada para el proyecto “Venom” se realizó el siguiente análisis basado en los objetivos principales del proyecto

- ¿Qué tan volátil es la definición de las capacidades de la IA?

Al trabajar con modelos de lenguaje, las capacidades de respuesta y los límites de procesamiento (tokens) se descubren durante la integración. Esto hace que los requerimientos funcionales del "Simbionte" sean altamente adaptables

- ¿Cuál es la complejidad de la sincronización de datos relacionales y vectoriales?

Existe un desafío técnico significativo en asegurar que la información en PostgreSQL coincida perfectamente con los *embeddings* de la plataforma, especialmente en un entorno donde cada empresa (tenant) debe tener sus datos aislados

- ¿existen riesgos asociados al uso de servidores de terceros?

La dependencia de APIs externas para la IA introduce riesgos de latencia, costos variables y posibles cambios en las políticas de privacidad de datos, lo que requiere un diseño arquitectónico flexible

- ¿influye la diversidad de datos de los clientes en las prioridades de desarrollo?

Dado que VENOM sirve a diferentes microempresas, una empresa puede priorizar la carga de documentos legales mientras otra prioriza manuales técnicos, obligando al sistema a ser independiente al tipo de contenido

Enfoque seleccionado

Nosotros hemos seleccionado un enfoque ágil como modelo de desarrollo incremental

- ¿Por qué se eligió este enfoque de desarrollo?

El enfoque incremental permite entregar primero el núcleo de seguridad y almacenamiento, y posteriormente añadir la capa de inteligencia de forma modular, lo cual una metodología rígida como lo puede ser una cascada no se podría realizar

- ¿Qué ventajas nos da este enfoque?

Nos permite obtener validaciones tempranas de la arquitectura de roles (multi-tenant) y asegura que la integración con la inteligencia artificial sea funcional antes de dinalizar el desarrollo de la interfaz de usuario

- ¿Qué riesgos evita?

Ayuda a reducir el impacto que pueda hacer fallos técnicos en la integración de módulos o funcionalidades al detectarlos en etapas tempranas

Marco de trabajo

Se ha seleccionado **Scrum** como marco de trabajo para la gestión del desarrollo, permitiendo una estructura de roles y eventos que garantizan la entrega de valor constante

Sprints: El desarrollo se divide en ciclos de dos (2) semanas, permitiendo una inspección y adaptación rápida de las funcionalidades de IA.

- Sprint Planning: Al inicio de cada ciclo, se seleccionan las Historias de Usuario del Product Backlog (ej. "Carga de PDFs", "Generación de Embeddings") que formarán el Sprint Backlog.
- Daily Scrum: Reuniones diarias de máximo 15 minutos para sincronizar el trabajo entre David Mora y Daniel González, identificando bloqueos técnicos en los contenedores Docker o en la lógica de FastAPI.

- **Sprint Review:** Al finalizar el sprint, se realiza una demostración del incremento funcional (ej. una consulta exitosa al RAG) ante el Gestor del Conocimiento.
- **Sprint Retrospective:** Espacio para analizar el desempeño del equipo y realizar ajustes en el flujo de trabajo para el siguiente ciclo.

Backlog

A partir de las historias de usuario derivadas de los objetivos y requerimientos del proyecto, se estructuró el Product Backlog y su distribución en el Sprint Backlog, siguiendo el marco de trabajo Scrum descrito en la sección de Metodología. Cada historia se priorizó y estimó en Story Points y fue asignada a uno de los seis sprints planificados de dos semanas cada uno.

Product Backlog

El Product Backlog agrupa la totalidad de las historias de usuario del proyecto VENOM, organizadas por épica y ordenadas por prioridad de negocio.

ID	Épica	Historia de Usuario	Prioridad	SP	Sprint
HU-01	Autenticación y Gestión de Usuarios (Multi-tenant)	Como administrador quiero registrar mi empresa en el sistema para poder gestionar mi propio tenant.	Alta	5	Sprint 1
HU-02	Autenticación y Gestión de Usuarios (Multi-tenant)	Como usuario quiero iniciar sesión con correo y contraseña para acceder de forma segura al sistema.	Crítica	3	Sprint 1
HU-03	Autenticación y Gestión de Usuarios (Multi-tenant)	Como administrador quiero asignar roles a los usuarios de mi empresa para controlar sus permisos.	Alta	5	Sprint 1
HU-04	Infraestructura y DevOps	Como desarrollador quiero configurar el entorno con Docker Compose para garantizar consistencia entre entornos.	Alta	5	Sprint 1
HU-05	Infraestructura y DevOps	Como desarrollador quiero declarar las dependencias en	Media	2	Sprint 1

		requirements.txt y package.json para asegurar reproducibilidad.			
HU-06	Ingesta y Vectorización de Documentos (RAG)	Como administrador quiero cargar documentos PDF/JSON al sistema para que la IA los use como base de conocimiento.	Crítica	8	Sprint 2
HU-07	Ingesta y Vectorización de Documentos (RAG)	Como sistema quiero fragmentar y vectorizar los documentos cargados para permitir búsquedas semánticas eficientes.	Crítica	8	Sprint 2
HU-08	Ingesta y Vectorización de Documentos (RAG)	Como administrador quiero categorizar los documentos cargados para controlar qué información puede consultar la IA.	Alta	5	Sprint 2
HU-09	Ingesta y Vectorización de Documentos (RAG)	Como administrador quiero ver el estado de vectorización de cada documento para saber si está listo para consulta.	Media	3	Sprint 2
HU-10	Motor de Consulta RAG / Chat con IA	Como usuario quiero hacer preguntas a la IA en un chat para obtener respuestas rápidas basadas en documentos.	Crítica	8	Sprint 3

HU-11	Motor de Consulta RAG / Chat con IA	Como sistema quiero filtrar los documentos por empresa y rol antes de responder para asegurar el aislamiento multi-tenant.	Crítica	8	Sprint 3
HU-12	Discriminación de Roles (System Prompts)	Como administrador quiero configurar el system prompt de mi empresa para personalizar el comportamiento de la IA.	Alta	5	Sprint 3
HU-13	Discriminación de Roles (System Prompts)	Como sistema quiero inyectar instrucciones distintas según el rol del usuario para diferenciar respuestas de clientes y empleados.	Alta	5	Sprint 3
HU-14	Extensión de Navegador (Frontend)	Como usuario quiero usar una extensión de navegador para consultar a la IA sin salir de mis herramientas de trabajo.	Crítica	8	Sprint 4
HU-15	Persistencia de Historial de Conversaciones	Como usuario quiero visualizar el historial de mis consultas para retomar conversaciones anteriores.	Media	5	Sprint 4

HU-16	Extensión de Navegador (Frontend)	Como usuario quiero cambiar entre roles interno/externo en la interfaz para adaptar mi experiencia.	Alta	3	Sprint 4
HU-17	Extensión de Navegador (Frontend)	Como desarrollador quiero validar los formularios en el frontend con TypeScript para evitar errores antes de llamar al backend.	Media	3	Sprint 4
HU-18	Conexión a Fuentes de Datos Externas (API/SQL)	Como usuario quiero que la IA consulte datos en tiempo real desde una API/SQL externa para conocer estados de servicio o inventarios.	Alta	8	Sprint 5
HU-19	Persistencia de Historial de Conversaciones	Como sistema quiero registrar cada conversación y mensaje en la base de datos para mantener el hilo del soporte.	Media	5	Sprint 5
HU-20	Seguridad, Privacidad y Aislamiento Multi-tenant	Como administrador quiero consultar los logs de auditoría para saber quién realizó qué consulta y cuándo.	Alta	5	Sprint 5

HU-21	Seguridad, Privacidad y Aislamiento Multi-tenant	Como sistema quiero anonimizar datos sensibles (PII) antes de enviarlos al LLM externo para proteger la privacidad.	Crítica	5	Sprint 5
HU-22	Pruebas y Calidad de Software	Como desarrollador quiero implementar pruebas unitarias en el backend para garantizar la fiabilidad de los servicios.	Alta	5	Sprint 6
HU-23	Pruebas y Calidad de Software	Como desarrollador quiero implementar pruebas end-to-end en el frontend para validar los flujos principales del sistema.	Alta	5	Sprint 6
HU-24	Pruebas y Calidad de Software	Como desarrollador quiero ejecutar pruebas de carga y estrés con JMeter para validar la escalabilidad del sistema.	Media	5	Sprint 6
HU-25	Motor de Consulta RAG / Chat con IA	Como usuario quiero calificar la utilidad de las respuestas de la IA para contribuir a mejorar el sistema de recuperación.	Baja	3	Sprint 6
HU-26	Infraestructura y DevOps	Como administrador quiero que el sistema mantenga una	Alta	8	Sprint 6

		disponibilidad del 99% para asegurar continuidad operativa.			
--	--	---	--	--	--

Sprint Backlog

El Sprint Backlog detalla la asignación de historias de usuario a cada uno de los seis sprints de dos semanas, junto con el responsable principal de cada historia dentro del equipo.

Sprint 1 - Fundamentos y Autenticación Multi-tenant

ID	Historia de Usuario	Prioridad	SP	Responsable
HU-01	Como administrador quiero registrar mi empresa en el sistema para poder gestionar mi propio tenant.	Alta	5	David Mora
HU-02	Como usuario quiero iniciar sesión con correo y contraseña para acceder de forma segura al sistema.	Crítica	3	Daniel González
HU-03	Como administrador quiero asignar roles a los usuarios de mi empresa para controlar sus permisos.	Alta	5	David Mora
HU-04	Como desarrollador quiero configurar el entorno con Docker Compose para garantizar consistencia entre entornos.	Alta	5	Daniel González
HU-05	Como desarrollador quiero declarar las dependencias en requirements.txt y package.json para asegurar reproducibilidad.	Media	2	David Mora

Sprint 2 - Ingesta y Vectorización de Documentos (RAG)

ID	Historia de Usuario	Prioridad	SP	Responsable
HU-06	Como administrador quiero cargar documentos PDF/JSON al sistema para que la IA los use como base de conocimiento.	Crítica	8	David Mora
HU-07	Como sistema quiero fragmentar y vectorizar los documentos cargados para permitir búsquedas semánticas eficientes.	Crítica	8	Daniel González
HU-08	Como administrador quiero categorizar los documentos cargados para controlar qué información puede consultar la IA.	Alta	5	David Mora
HU-09	Como administrador quiero ver el estado de vectorización de cada documento para saber si está listo para consulta.	Media	3	Daniel González

Sprint 3 - Motor de Consulta RAG y Roles

ID	Historia de Usuario	Prioridad	SP	Responsable
HU-10	Como usuario quiero hacer preguntas a la IA en un chat para obtener respuestas rápidas basadas en documentos.	Crítica	8	David Mora

HU-11	Como sistema quiero filtrar los documentos por empresa y rol antes de responder para asegurar el aislamiento multi-tenant.	Crítica	8	Daniel González
HU-12	Como administrador quiero configurar el system prompt de mi empresa para personalizar el comportamiento de la IA.	Alta	5	David Mora
HU-13	Como sistema quiero inyectar instrucciones distintas según el rol del usuario para diferenciar respuestas de clientes y empleados.	Alta	5	Daniel González

Sprint 4 - Extensión de Navegador (Frontend)

ID	Historia de Usuario	Prioridad	SP	Responsable
HU-14	Como usuario quiero usar una extensión de navegador para consultar a la IA sin salir de mis herramientas de trabajo.	Crítica	8	David Mora
HU-15	Como usuario quiero visualizar el historial de mis consultas para retomar conversaciones anteriores.	Media	5	Daniel González
HU-16	Como usuario quiero cambiar entre roles interno/externo en la interfaz para adaptar mi experiencia.	Alta	3	David Mora
HU-17	Como desarrollador quiero validar los formularios en el frontend con TypeScript para evitar errores antes de llamar al backend.	Media	3	Daniel González

Sprint 5 - Fuentes Externas y Persistencia de Historial

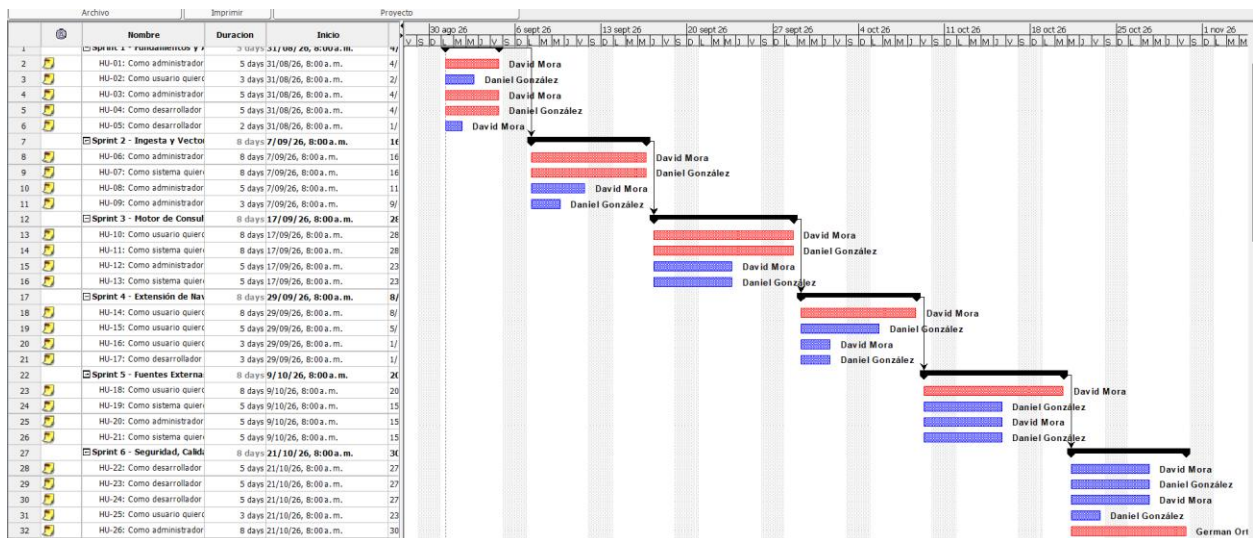
ID	Historia de Usuario	Prioridad	SP	Responsable
HU-18	Como usuario quiero que la IA consulte datos en tiempo real desde una API/SQL externa para conocer estados de servicio o inventarios.	Alta	8	David Mora
HU-19	Como sistema quiero registrar cada conversación y mensaje en la base de datos para mantener el hilo del soporte.	Media	5	Daniel González
HU-20	Como administrador quiero consultar los logs de auditoría para saber quién realizó qué consulta y cuándo.	Alta	5	David Mora
HU-21	Como sistema quiero anonimizar datos sensibles (PII) antes de enviarlos al LLM externo para proteger la privacidad.	Crítica	5	Daniel González

Sprint 6 - Seguridad, Calidad y Cierre

ID	Historia de Usuario	Prioridad	SP	Responsable
HU-22	Como desarrollador quiero implementar pruebas unitarias en el backend para garantizar la fiabilidad de los servicios.	Alta	5	David Mora

HU-23	Como desarrollador quiero implementar pruebas end-to-end en el frontend para validar los flujos principales del sistema.	Alta	5	Daniel González
HU-24	Como desarrollador quiero ejecutar pruebas de carga y estrés con JMeter para validar la escalabilidad del sistema.	Media	5	David Mora
HU-25	Como usuario quiero calificar la utilidad de las respuestas de la IA para contribuir a mejorar el sistema de recuperación.	Baja	3	Daniel González
HU-26	Como administrador quiero que el sistema mantenga una disponibilidad del 99% para asegurar continuidad operativa.	Alta	8	German Ortiz Diaz (Gestor del Conocimiento)

Projectlibre



Requerimientos

Requerimientos funcionales

ID	Requerimiento	Descripción	Importancia
RF-01	Ingesta RAG	El sistema debe procesar documentos (PDF/SQL/JSON) y convertirlos en vectores consultables.	Crítica
RF-02	Interfaz de Chat	Proporcionar una ventana de interacción fluida dentro de la extensión de navegador.	Crítica
RF-03	Discriminación de Roles	El backend debe inyectar instrucciones distintas (System Prompts) según si el usuario es cliente o empleado.	Alta
RF-04	Conexión a Base de Datos	La IA debe poder consultar datos en tiempo real de una base de datos SQL (ej. estados de cuenta).	Alta
RF-05	Persistencia de Historial	Almacenar las conversaciones previas para mantener el hilo conductor del soporte.	Media

Requerimientos no funcionales

ID	Requerimiento	Atributo	Descripción	Importancia
----	---------------	----------	-------------	-------------

RNF-01	Latencia	Rendimiento	El tiempo de respuesta de la IA no debe superar los 3.5 segundos por consulta.	Crítica
RNF-02	Seguridad PII	Privacidad	El sistema debe anonimizar datos sensibles antes de enviarlos a la API del modelo de lenguaje.	Crítica
RNF-03	Disponibilidad	Fiabilidad	La Capa de Inteligencia debe estar operativa el 99% del tiempo (High Availability).	Alta
RNF-04	Escalabilidad	Capacidad	El backend debe soportar múltiples clientes (multi-tenant) sin mezclar sus bases de datos.	Alta
RNF-05	Compatibilidad	Portabilidad	La extensión debe ser compatible con navegadores basados en Chromium (Chrome, Edge, Brave).	Media

Dependencias

Tipo	Dependencia	Descripción	Importancia
API Externa	Modelos de Lenguaje (LLM)	Acceso a las APIs de OpenAI (GPT-4o) o Anthropic (Claude) como cerebro del sistema.	Crítica
Infraestructura	Base de Datos Vectorial	Servicio en la nube (ej. Pinecone) para el almacenamiento de los "conocimientos" de la empresa.	Crítica
Librerías	Framework LangChain	Framework esencial para orquestar la lógica entre la IA, los documentos y el usuario.	Alta
Entorno	Runtime de Python/Node	Necesidad de servidores con soporte para Python 3.10+ y Node.js para el build del frontend.	Alta

Configuración tecnológica

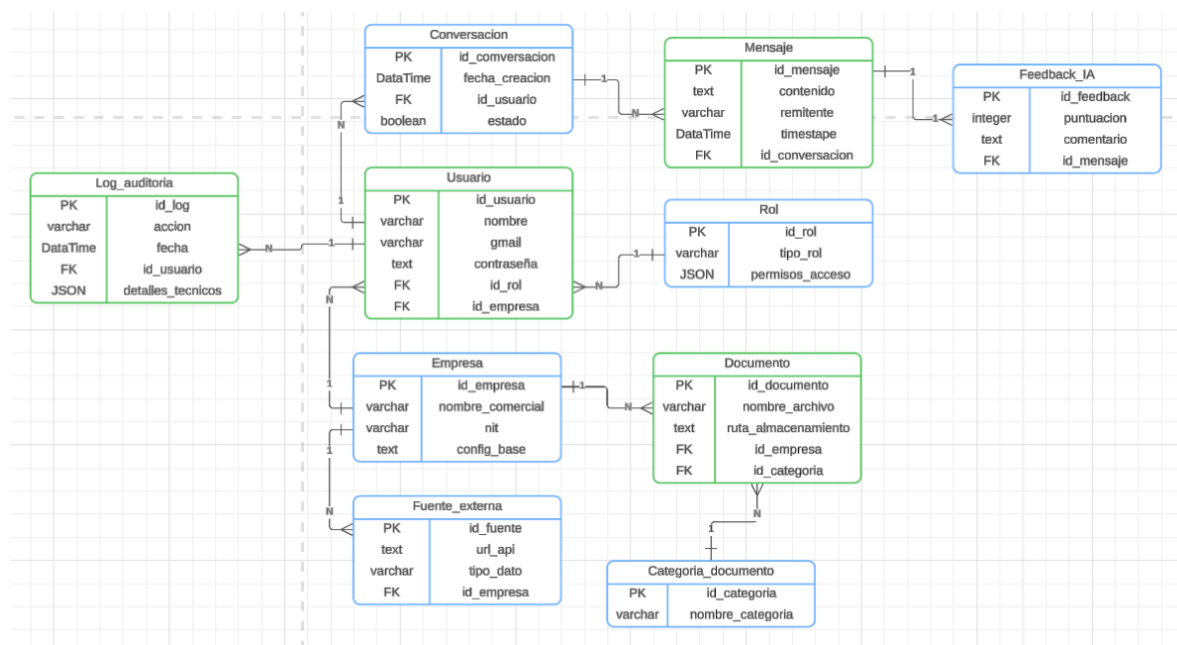
Capa	Tecnología
Frontend	React
Backend	Phyton (FastAPI)
Base de datos	postgreSQL
Control de versiones	Git

Riesgos

Riesgo	Descripción	Mitigación	Nivel
Alucinaciones de la IA	La IA genera respuestas que parecen coherentes pero son incorrectas o simplemente inventadas	Implementación de arquitectura RAG para así obligar a la IA a solo basarse en los documentos cargados y historial de chats guardados	Alto
Falla de aislamientos multi-tenant	Riesgo de que la empresa pueda acceder a la base de conocimiento de otra organización	Validación específica y regulada del id de la compañía en todas las consultas de la base de datos	Critico
Latencia de procesamiento RAG	El tiempo de respuesta entre la consulta del usuario y la respuesta de la IA sea superior al limite de usabilidad	Uso de funciones en FastAPI que sean asíncronas y optimización de índices en la búsqueda vectorial para agilizar la respuesta	Medio

Agotamiento de tokens de API	Interrupción del servicio de IA por exceder el limite de tokens	Implementación de un sistema que monitoree el consumo por empresa y limites de uso configurables en el backend	Alto
Inconsistencia de entornos	El sistema funciona en desarrollo pero presenta errores al ser desplegado debido a diferentes versiones de dependencias y librerías	Uso de Docker y archivos de requerimientos bloqueados para garantizar un despliegue total entre desarrollo y producción	Medio

Base de datos



Descripción tablas

- Empresa: Es la entidad raíz que permite el manejo de múltiples clientes. Almacena los datos legales y la configuración personalizada de la IA para que el sistema sepa cómo comportarse específicamente para cada organización
- Rol: Define los niveles de privilegio dentro de la plataforma (como Administrador, Agente o Cliente). Es fundamental para la "discriminación lógica", asegurando que cada usuario acceda solo a las funciones y datos que le corresponden
- Usuario: Contiene la información de perfil y las credenciales de acceso de todas las personas que interactúan con el sistema, vinculándolas obligatoriamente a una empresa y a un rol específico
- Categoria_Documento: Funciona como un clasificador de seguridad y organización. Permite agrupar los archivos (por ejemplo: "Manuales Técnicos" o "Políticas Públicas") para controlar qué información puede consultar la IA según el contexto que da el usuario
- Documento: Gestiona los metadatos de los archivos cargados al sistema. Registra el estado de procesamiento (si ya fue vectorizado o si hubo un error) antes de que la información pase a la base de datos
- Conversacion: Actúa como un contenedor para las sesiones de chat. Su propósito es agrupar los mensajes para mantener el hilo conductor y permitir que la IA tenga "memoria" de lo que se ha hablado anteriormente en esa sesión específica
- Mensaje: Es el registro individual de cada interacción. Almacena el contenido de la pregunta del usuario o la respuesta generada por la IA, junto con la hora exacta de la comunicación
- Log_Auditoria: Es la tabla dedicada a la seguridad y trazabilidad. Registra cada acción importante realizada en el sistema, como quién realizó una consulta o cargó un archivo

- **Fuente_Externa:** Permite la "simbiosis de datos" al configurar conexiones con servicios externos (como APIs de inventarios o bases de datos SQL). Esto permite que la IA consulte información externa en tiempo real para dar respuestas más precisas
- **Feedback_IA:** Proporciona un mecanismo de mejora continua. Permite a los usuarios calificar la utilidad de las respuestas de la IA, generando datos valiosos para ajustar los algoritmos de recuperación de información (RAG)

Diccionario de datos

Tabla empresa

Atributo	Tipo de Dato	Descripción
id_empresa	Serial (PK)	Identificador único y llave primaria de la organización
nombre_empresa	Varchar(100)	Nombre legal o comercial de la empresa cliente
nit	Varchar(20)	Número de identificación tributaria (único)
config_ai_prompt	Text	Instrucciones personalizadas (System Prompt) para el comportamiento de la IA

Tabla usuario

Atributo	Tipo de Dato	Descripción
id_usuario	Serial (PK)	Identificador único del usuario
correo	Varchar(150)	Correo electrónico (utilizado para el login y tokens)
password_hash	Text	Contraseña encriptada mediante algoritmos de hashing
id_rol	Integer (FK)	Referencia a la tabla de Roles para permisos

id_empresa	Integer (FK)	Llave foránea que garantiza que el usuario solo vea datos de su empresa
------------	--------------	---

Tabla documento

Atributo	Tipo de Dato	Descripción
id_documento	Serial (PK)	Identificador único del archivo
nombre_archivo	Varchar(255)	Nombre original del documento cargado
estado_vectorizado	Boolean	Indica si el contenido ya fue procesado y guardado en la base de datos
fecha_carga	Timestamp	Registro de cuándo se integró la información al sistema
id_empresa	Integer (FK)	Llave foránea para asegurar que la IA no recupere documentos de otros clientes

Tabla mensaje

Atributo	Tipo de Dato	Descripción
id_mensaje	Serial (PK)	Identificador único del mensaje
contenido	Text	Texto de la pregunta o respuesta generada
emisor	Enum	Define si el mensaje fue creado por el 'Usuario' o por la 'IA'
id_conversacion	Integer (FK)	Vincula el mensaje a una sesión de chat específica

Tabla rol

Atributo	Tipo de Dato	Descripción
id_rol	Serial (PK)	Identificador único del nivel de acceso
nombre_rol	Varchar(50)	Etiqueta del rol (ej: 'EmpresaAdmin', 'Operativo').
descripcion	Text	Explicación de los permisos asociados a este rol

Tabla conversación

Atributo	Tipo de Dato	Descripción
id_conversacion	Serial (PK)	Identificador único de la sesión de chat
titulo_temp	Varchar(200)	Título generado por la IA para resumir la charla
id_usuario	Integer (FK)	Identifica qué usuario inició la charla
fecha_creacion	Timestamp	Fecha y hora de inicio de la sesión

Diagrama de flujo

Ingesta y vectorización de datos

El Administrador carga un archivo; el sistema valida su tamaño, lo fragmenta y lo convierte en vectores (datos numéricos) que la IA guarda como conocimiento para poder consultarlos después



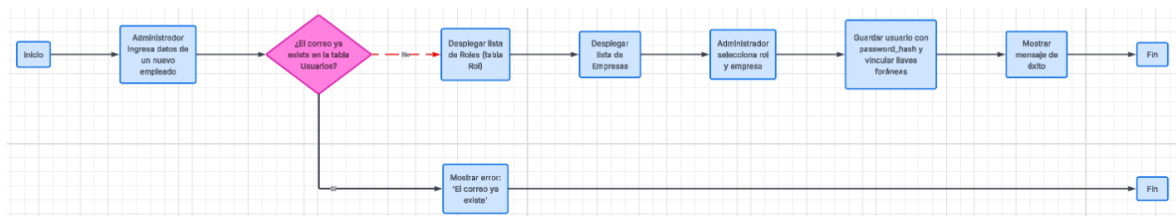
Consulta contextual (RAG)

El usuario pregunta; el sistema filtra los documentos por empresa y rol, extrae la información relevante y la IA genera una respuesta precisa y segura basada únicamente en ese contexto



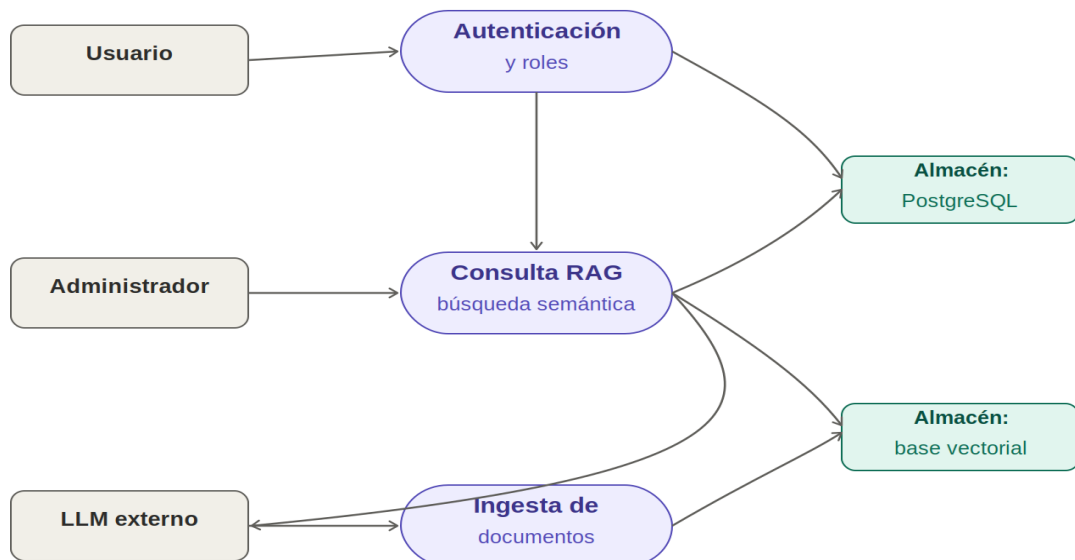
Administración multitenant

Controla el acceso al sistema asegurando que cada usuario esté vinculado a una empresa y un rol específico, garantizando que la información de una organización esté totalmente aislada de las demás



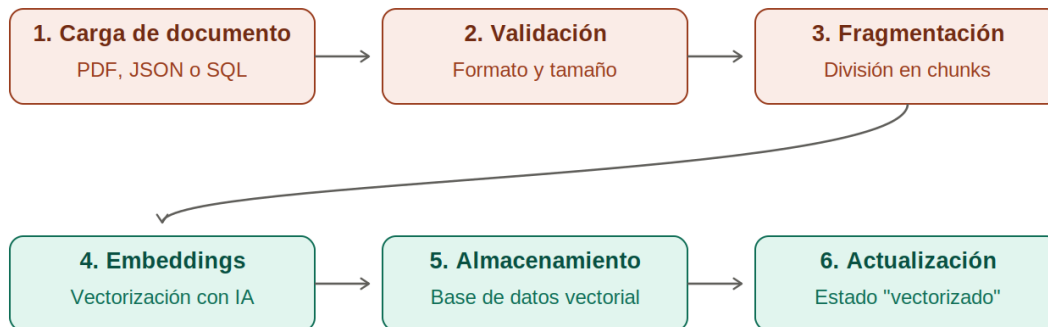
Flujo de datos

Este diagrama resume el flujo de datos general del sistema: las entidades externas (Usuario, Administrador y el LLM externo) interactúan con los procesos de Autenticación, Consulta RAG e Ingesta de documentos, los cuales leen y escriben en los almacenes PostgreSQL y la base de datos vectorial.



Proceso de datos

El siguiente pipeline detalla, a nivel técnico, el proceso interno que ejecuta el Document Service dentro del RAG Engine para transformar un documento cargado en conocimiento consultable por la IA.



Casos de uso

Para el diseño funcional de VENOM, se han identificado las interacciones clave entre los usuarios y el sistema, centradas en la gestión de conocimiento y la seguridad multi-tenant. Los casos de uso detallan la lógica de operación desde la administración de la plataforma hasta la consulta interactiva con la IA

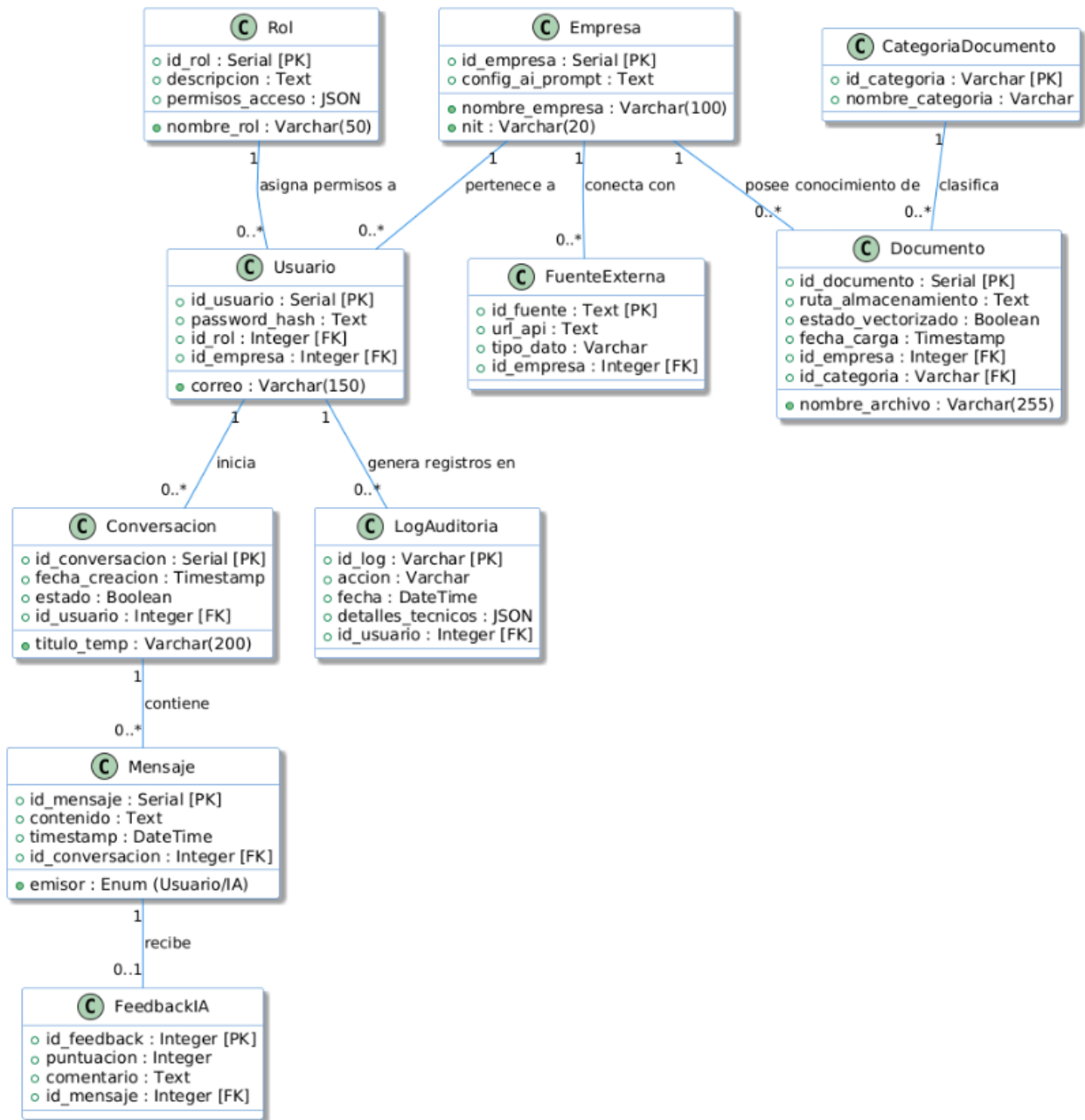
Anexo documento de casos de uso [Casos de uso.docx](#)

Historia de usuario

Las siguientes historias representan las necesidades de los usuarios y guían el desarrollo de los Sprints bajo el marco de trabajo Scrum

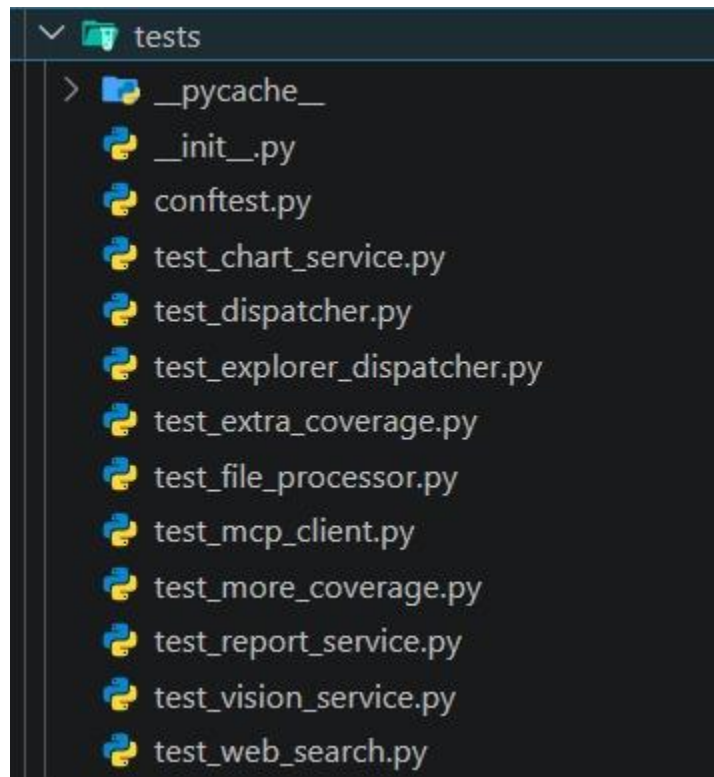
Anexo documento de historia de usuario [Historias de Usuario VENOM.docx](#)

Diagrama uml



Pruebas

Unitarias



Nota: dentro de la carpeta de venom-backend se creo una nueva carpeta llamada test que contiene todas las pruebas necesarias de los servicios implementados en VENOM

```

$ docker exec venom_backend pytest tests/ -v --cov=app

----- coverage: platform linux, python 3.11.15-final-0 -----
Name                                                                    Stmts   Miss  Cover
-----
app/__init__.py                                                            0      0   100%
app/database.py                                                            12     12     0%
app/services/audit_service.py                                             39      7    82%
app/services/chart_service.py                                            99     52    47%
app/services/dispatchers/__init__.py                                     37      3    92%
app/services/dispatchers/audit_dispatcher.py                             15      2    87%
app/services/dispatchers/base.py                                          15      2    87%
app/services/dispatchers/chart_dispatcher.py                             56     22    61%
app/services/dispatchers/compare_documents_dispatcher.py                 34     10    71%
app/services/dispatchers/email_dispatcher.py                             18      3    83%
app/services/dispatchers/explorer_dispatcher.py                          18      2    89%
app/services/dispatchers/legacy_dispatcher.py                           19      4    79%
app/services/dispatchers/list_documents_dispatcher.py                    21      2    90%
app/services/dispatchers/mcp_apps_dispatcher.py                          18      4    78%
app/services/dispatchers/mcp_music_dispatcher.py                        18      4    78%
app/services/dispatchers/mcp_system_dispatcher.py                       21      7    67%
app/services/dispatchers/mcp_windows_dispatcher.py                      18      4    78%
app/services/dispatchers/pdf_dispatcher.py                               27      2    93%
app/services/dispatchers/process_file_dispatcher.py                      22      6    73%
app/services/dispatchers/web_search_dispatcher.py                        18      4    78%
app/services/document_comparison_service.py                               25      5    80%
app/services/email_service.py                                             72     54    25%
app/services/file_processor.py                                           122     57    53%
app/services/mcp_client.py                                                91     51    44%
app/services/report_service.py                                            88     43    51%
app/services/vision_service.py                                            69     34    51%
app/services/web_search_service.py                                        52     36    31%
-----
TOTAL                                                                    1044    432    59%

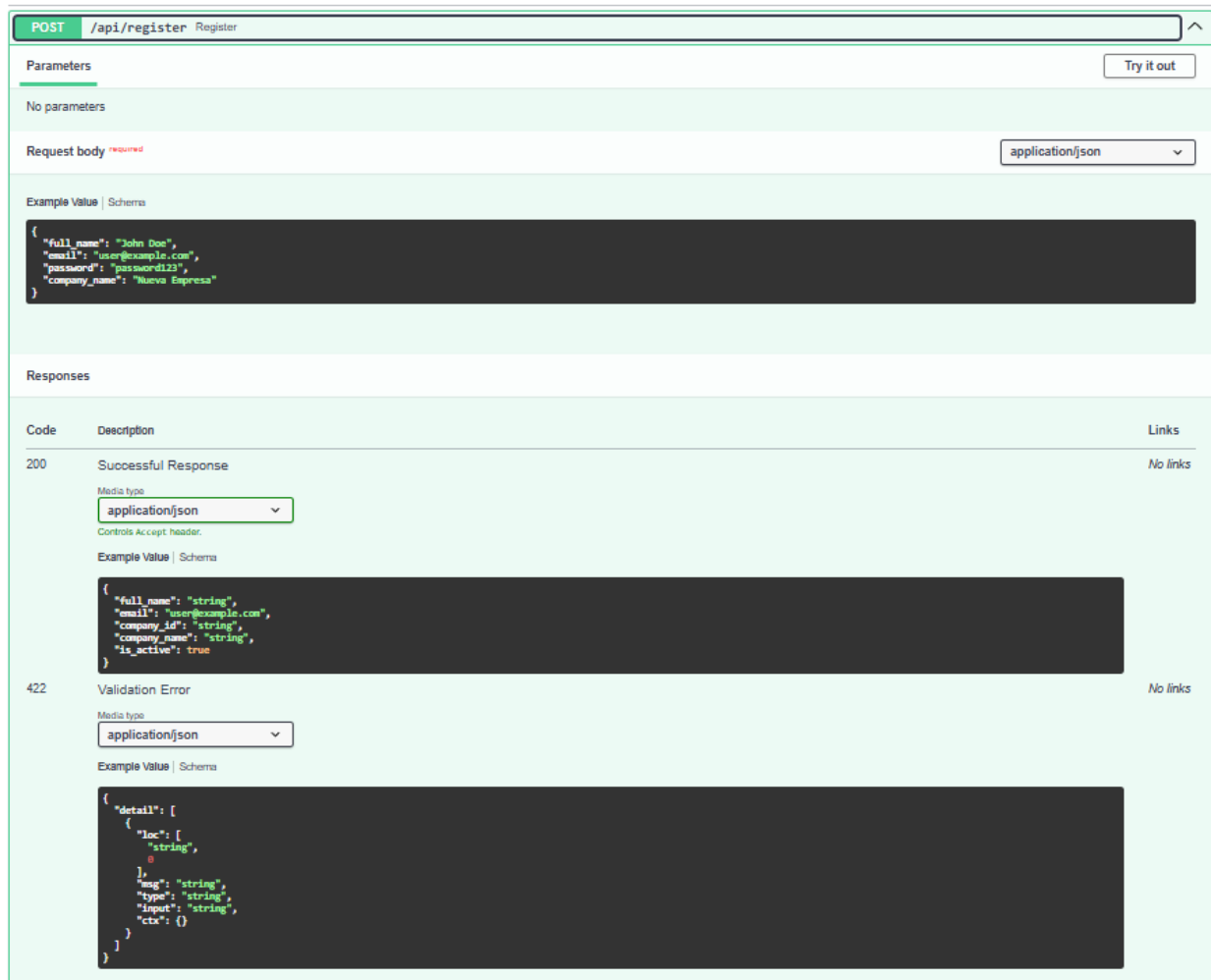
===== short test summary info =====
FAILED tests/test_extra_coverage.py::test_mcp_system_dispatcher - assert ("Er...
FAILED tests/test_more_coverage.py::test_email_send_missing_config - assert F...
===== 2 failed, 38 passed, 2 skipped, 56 warnings in 4.48s =====

```

Nota: al ejecutar el script de las pruebas unitarias se pueden evidenciar con mayor detalle las pruebas implementadas, dentro de ese resultado se puede evidenciar el porcentaje y alcance de cobertura que tienen estas la cual muestra que 13 pasaron, 2 fueron saltadas y 2 falladas

completando así un porcentaje de 59% en total

Postman



Nota: por medio de la plataforma swagger se puede evidenciar el api implementado para registrar evidenciando los espacios diseñados para poder configurar los parámetros requeridos

POST /api/login Login

Parameters

No parameters

Request body required application/x-www-form-urlencoded

```
grant_type
string | (string |
null)
pattern: "password$

username * required
string

password * required
string($password)

scope
string

client_id
string | (string |
null)

client_secret
string | (string |
null)($password)
```

Responses

Code	Description	Links
200	Successful Response	No links
	Media type application/json	
	Controls Accept header	
	Example Value Schema	
	<pre>"string"</pre>	
422	Validation Error	No links
	Media type application/json	
	Example Value Schema	
	<pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }] }</pre>	

Nota: por medio del swagger se puede evidenciar el api de login que sirve para poder reemplazar los campos que se establecieron como son email y contraseña que previamente se implementaron en el registro

POST /api/ingest/file Ingest File

Parameters

No parameters

Request body required

multipart/form-data

file * required
string

Responses

Code	Description	Links
200	Successful Response Media type: application/json Controls: Accept header. Example Value Schema <pre>"string"</pre>	No links
422	Validation Error Media type: application/json Example Value Schema <pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }]}</pre>	No links

Nota: por medio de swagger muestra el api el cual es el proceso de como se sube el documento

POST /api/ask/{conversation_id} Chat Multimodal

Parameters

Try it out

Name	Description
conversation_id required	conversation_id
string (path)	

Request body required

multipart/form-data

query required
string

file
string | (string
| null)

Responses

Code	Description	Links
200	Successful Response	No links
	Media type application/json	
	Controls Accept header.	
	Example Value Schema	
	<pre>"string"</pre>	
422	Validation Error	No links
	Media type application/json	
	Example Value Schema	
	<pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }] }</pre>	

Nota: por medio de swagger muestra el api de chat el cual es toda la lógica de consulta por medio de toda la inteligencia artificial con todos los servicios implementados en el proyecto

End to end



Nota: dentro del directorio de venom-frontend se agregó una nueva carpeta end to end la cual posee la lógica del flujo de los procesos principales

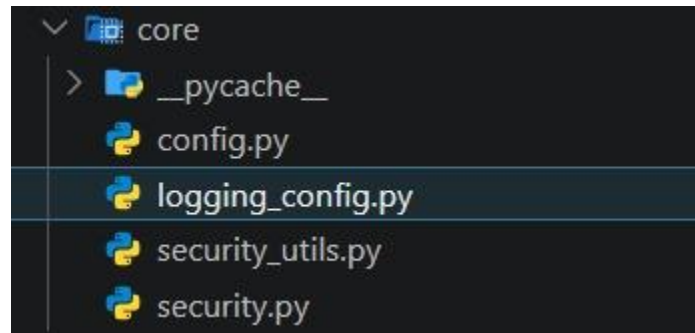
```
playwright.config.cjs U X
venom-frontend > playwright.config.cjs > <unknown>
1  // playwright.config.cjs
2  const { defineConfig } = require('@playwright/test');
3
4  module.exports = defineConfig({
5    testDir: './e2e',
6    timeout: 60000,
7    use: {
8      baseURL: 'http://localhost:5173',
9      headless: false,
10     viewport: { width: 1280, height: 720 },
11   },
12   // webServer: {
13     // command: 'npm run dev',
14     // port: 5173,
15     // reuseExistingServer: true,
16   },
17 });
```

Nota: la configuración para el flujo end to end el cual indica el puerto de react para el uso optimo de frontend y realizar los procesos establecidos

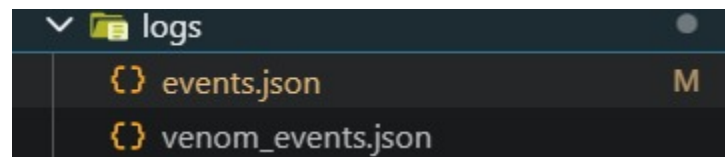
```
$ npx playwright test --headed  
  
Running 1 test using 1 worker  
  
✓ 1 e2e\venom-flow.spec.js:6:1 › Flujo completo de negocio VENOM (11.6s)  
  
1 passed (14.7s)
```

Nota: se evidencia la ejecución del script y en cuanto tiempo se completó sin ningún error

Logs



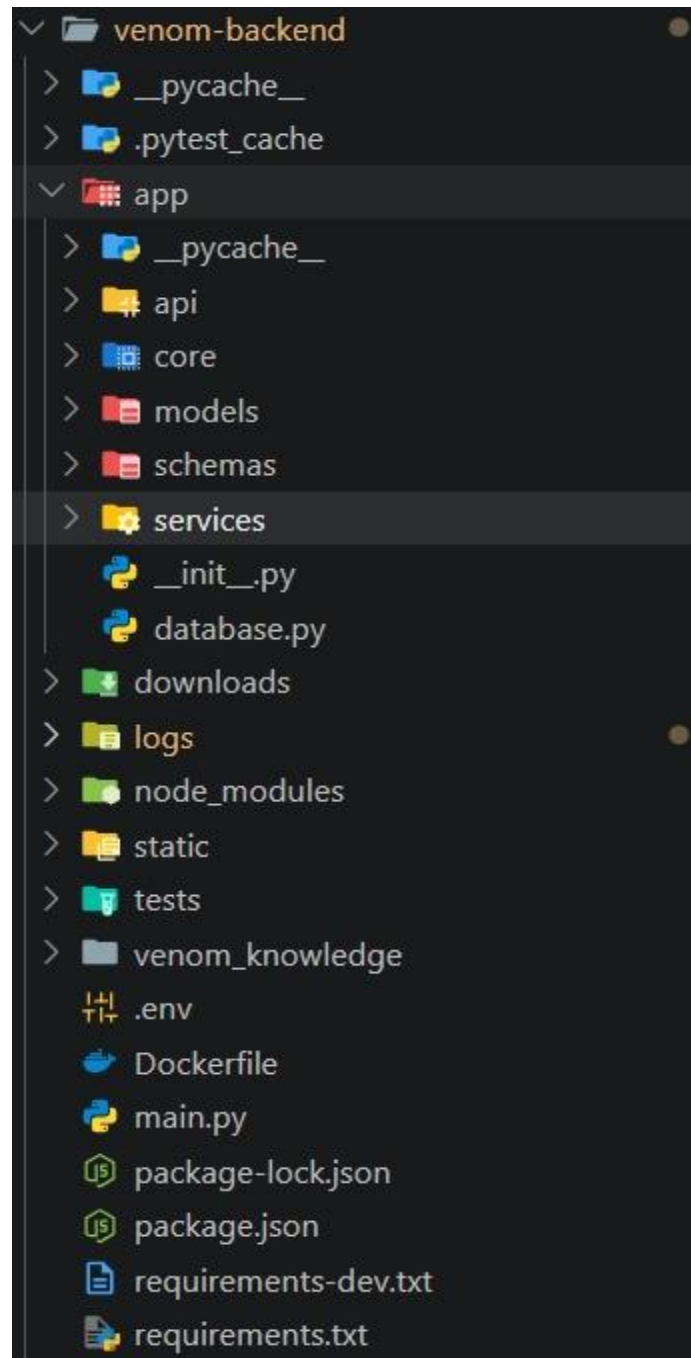
Nota: el código fuente de como se configura los logs para posteriormente guardarlos en la carpeta de logs



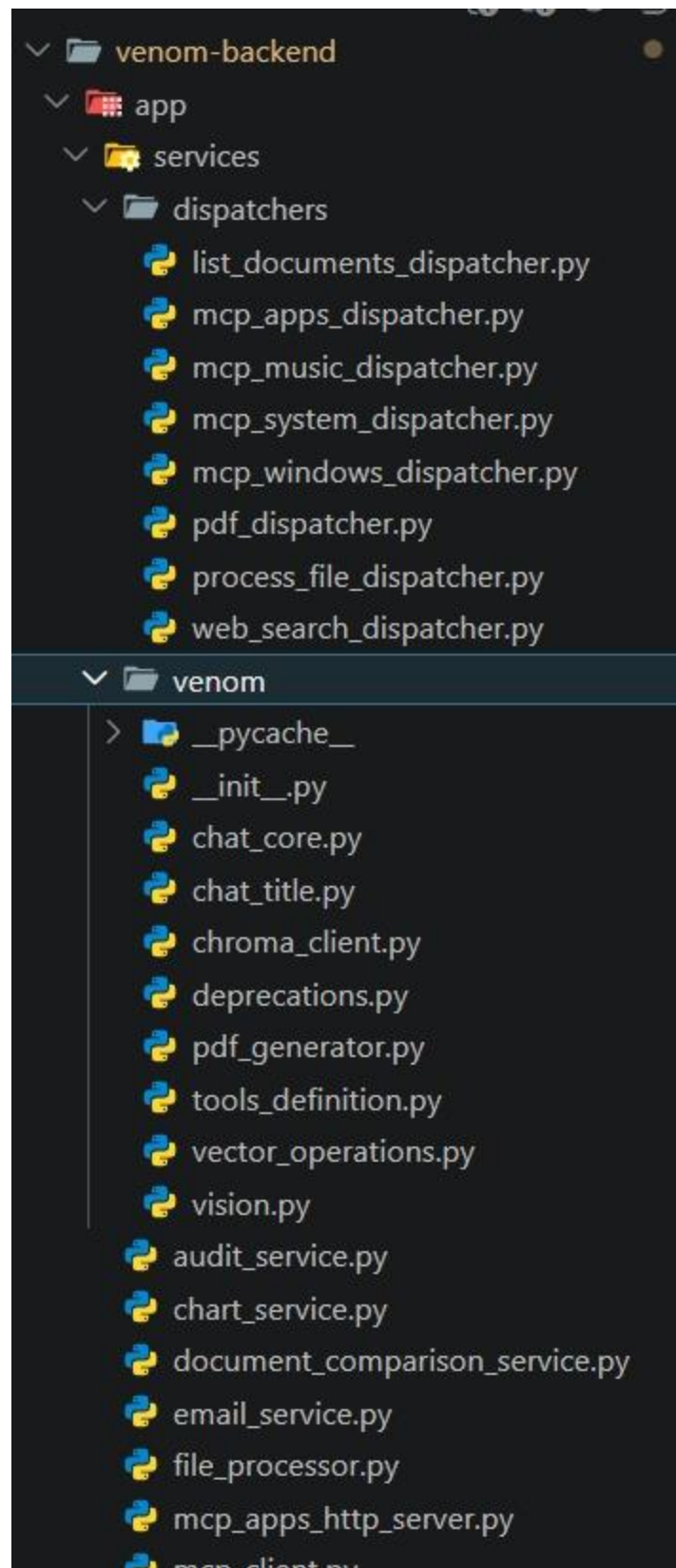
Nota: dentro de esta carpeta por medio de json se guardan todos los logs establecidos en el proyecto

principios solid

S – single responsibility principle



Nota: gracias a la arquitectura utilizada se utilizó el principio de responsabilidad única evidenciando en los distintos directorios donde cada uno está vinculada a una función o una responsabilidad específica



Nota: en este anexo se evidencia mejor el principio de responsabilidad única ya que se dividió mejor la responsabilidad de los servicios a los que están vinculado el proyecto

O – open/closed principle

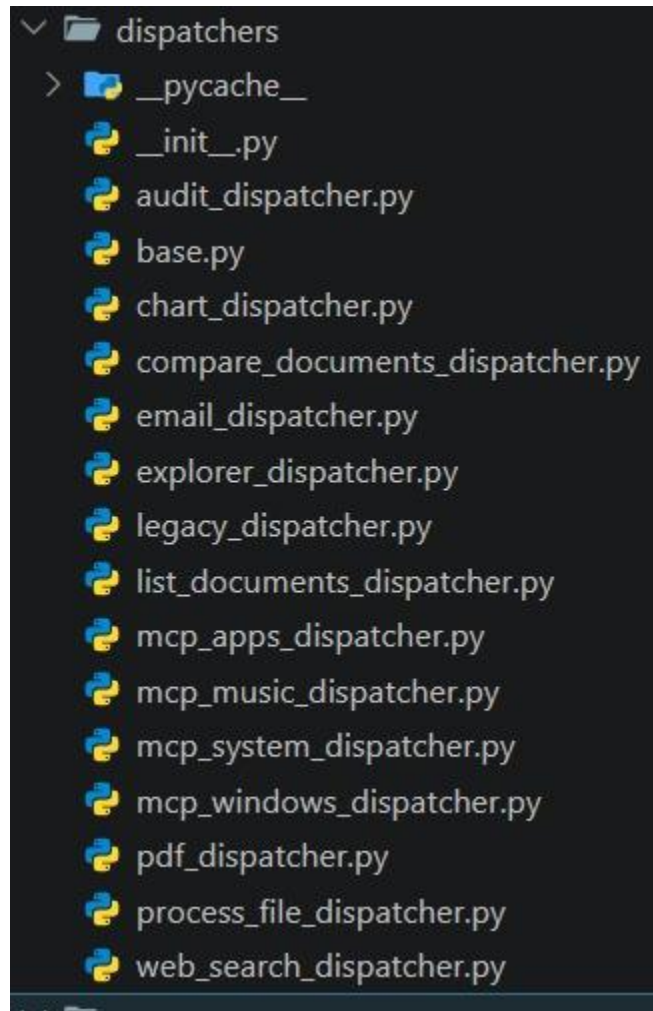
```
# ===== 0. AUTENTICACIÓN =====

@router.post("/register", response_model=schemas.UserResponse)
async def register(user_in: schemas.UserCreate, db: Session = Depends(get_db)):
    user_exists = db.query(models.User).filter(models.User.email == user_in.email).first()
    if user_exists:
        raise HTTPException(status_code=400, detail="El email ya está registrado.")
    try:
        company_name = getattr(user_in, 'company_name', f"Corporación {user_in.full_name}")
        new_company = models.Company(name=company_name)
        db.add(new_company)
        db.flush()
        hashed_pw = security.get_password_hash(user_in.password)
        new_user = models.User(
            email=user_in.email,
            full_name=user_in.full_name,
            hashed_password=hashed_pw,
            company_id=new_company.id,
            is_active=True
        )
        db.add(new_user)
        db.commit()
        db.refresh(new_user)
        return new_user
    except Exception as e:
        db.rollback()
        raise HTTPException(status_code=500, detail=f"Error en registro: {str(e)}")

@router.post("/login")
async def login(form_data: OAuth2PasswordRequestForm = Depends(), db: Session = Depends(get_db)):
    user = db.query(models.User).filter(models.User.email == form_data.username).first()
    if not user or not security.verify_password(form_data.password, user.hashed_password):
        raise HTTPException(status_code=401, detail="Credenciales incorrectas")
    access_token = security.create_access_token(data={"sub": user.email})
    refresh_token = security.create_refresh_token(email=user.email)
    return {
        "access_token": access_token,
```

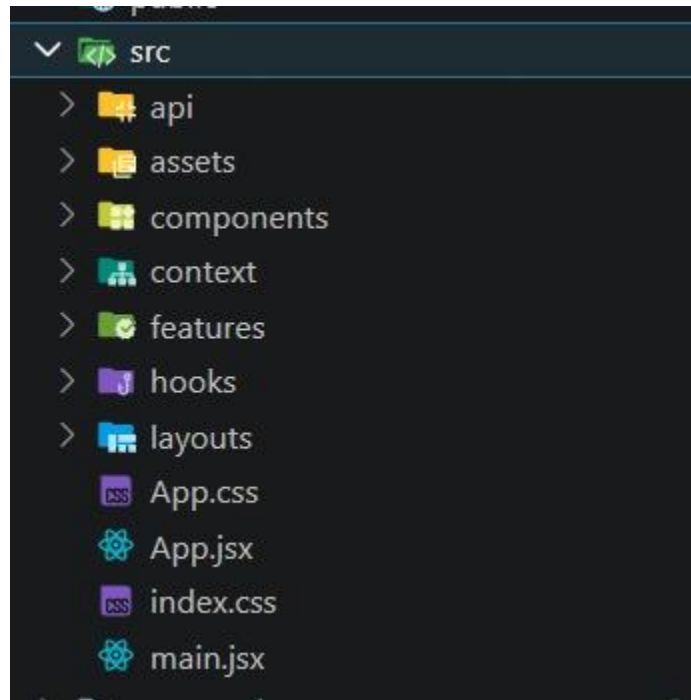
Nota: al dividir las responsabilidades y a su vez con los end points bien estructurado el proyecto es capas de extenderse agregando nuevas modificaciones o eliminando algunas sin tener que alterar el código ya existente

L - Liskov Substitution Principle



Nota: acá se evidencia como base.py define métodos que todas los handlers tiene que implementar, posteriormente se puede ver como las clases hijas tienen su propia lógica cada una

I - Interface Segregation Principle



Nota: en este anexo se evidencia como la interfaz esta dividida en diferentes interfaces la cual cada una se enfoca en una responsabilidad del proyecto por lo que no depende de una interfaz general

D - Dependency Inversion Principle

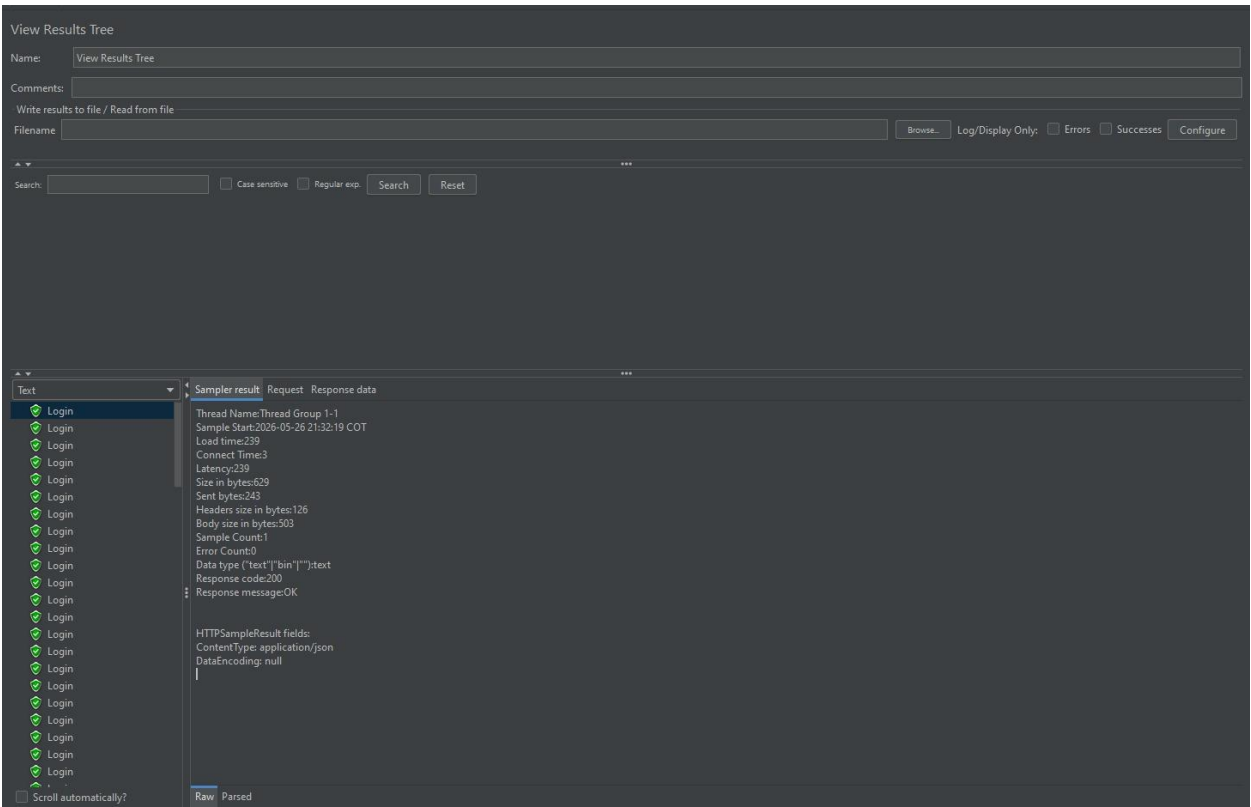
```
class VenomDispatcher:
    def __init__(self):
        self.handlers = [
            PdfDispatcher(),
            WebSearchDispatcher(),
            EmailDispatcher(),
            ProcessFileDispatcher(),
            ChartDispatcher(),
            CompareDocumentsDispatcher(),
            AuditDispatcher(),
            ListDocumentsDispatcher(),
            ExplorerDispatcher(),
            McpMusicDispatcher(),
            McpAppsDispatcher(),
            McpSystemDispatcher(),
            McpWindowsDispatcher(),
            LegacyMcpDispatcher(),
        ]
```

Nota: acá se puede ver como en la clase `VenomDispatcher` no depende directamente de las clase concretas, sino de la lista `handlers` en `base.py` que se mostro previamente en los principios de

liskov

Implementación de pruebas de carga y estrés (JMeter)

Implementación:



Nota: elaboración propia

Summary Report

Name: Summary Report

Comments:

Write results to file / Read from file

Filename: Log/Display Only: ☐ Errors ☐ Successes

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
Login	500	1301	6	4115	1734.12	80.00%	22.0/min	0.28	0.07	777.2
TOTAL	500	1301	6	4115	1734.12	80.00%	22.0/min	0.28	0.07	777.2

Nota: elaboración propia

Aggregate Report

Name: Aggregate Report

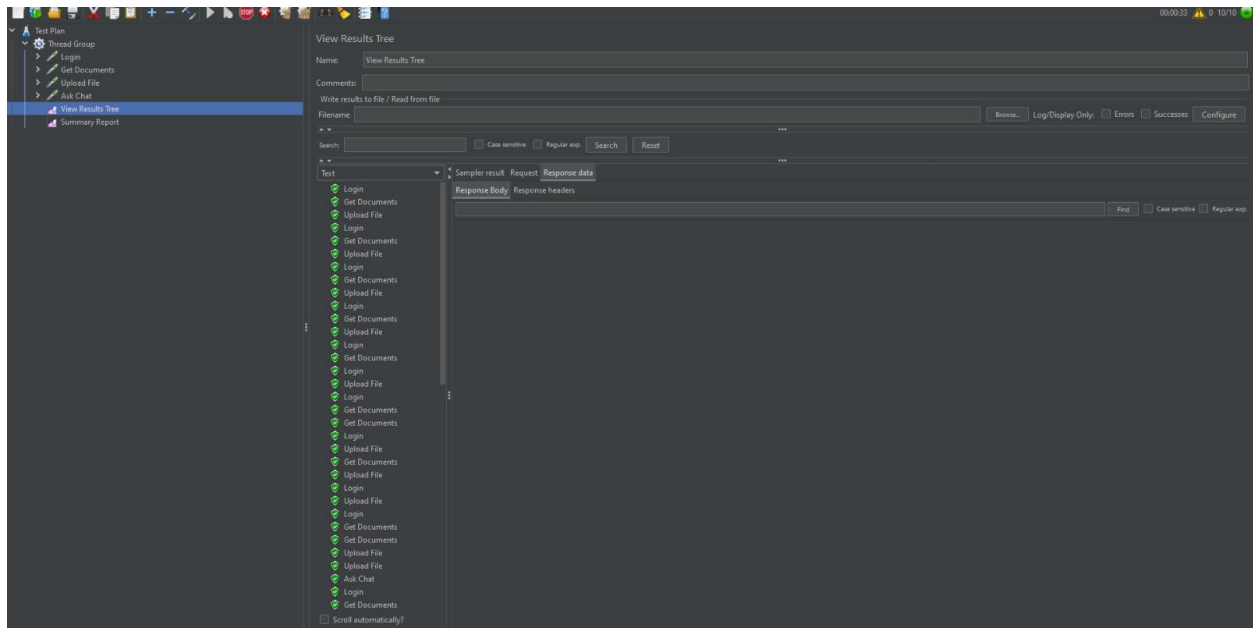
Comments:

Write results to file / Read from file

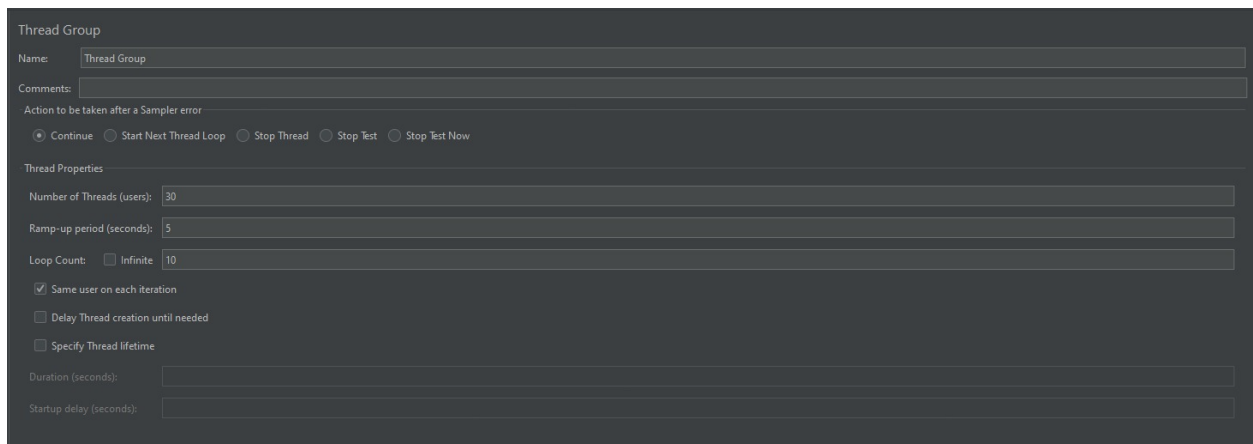
Filename: Log/Display Only: ☐ Errors ☐ Successes

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughput	Received KB/sec	Sent KB/sec
Login	500	1301	12	4094	4103	4109	6	4115	80.00%	22.0/min	0.28	0.07
TOTAL	500	1301	12	4094	4103	4109	6	4115	80.00%	22.0/min	0.28	0.07

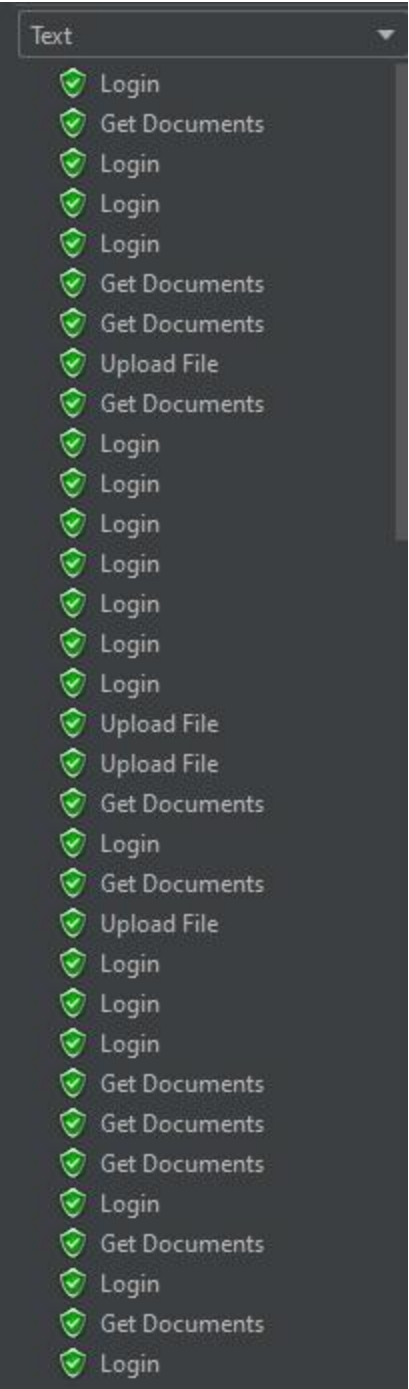
Nota: elaboración propia



Nota: elaboración propia



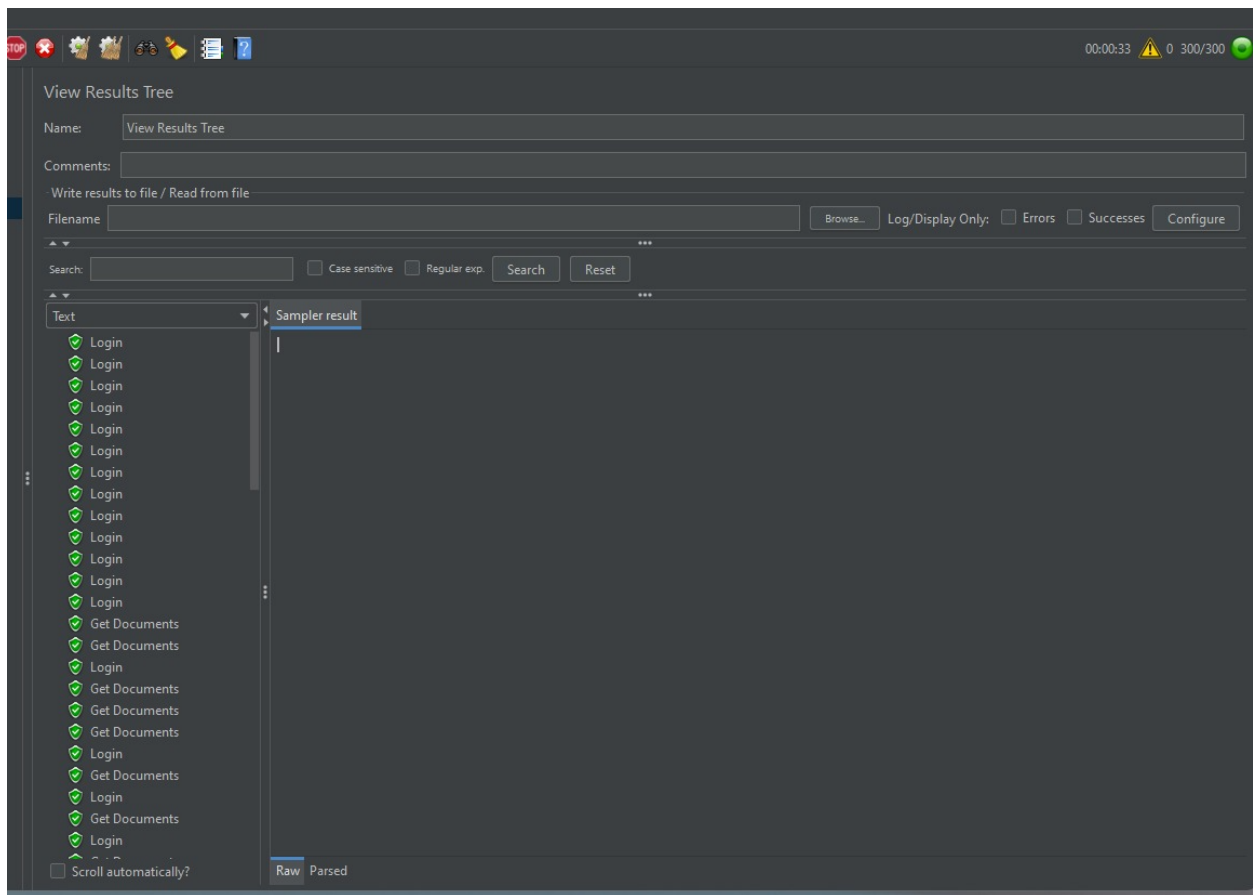
Nota: elaboración propia



Nota: elaboración propia

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
Login	109	26069	317	183417	53208.97	10.09%	59.3/hour	0.01	0.00	803.1
Get Documents	99	13032	9	127004	34972.86	20.20%	54.8/hour	0.09	0.00	5944.9
Upload File	97	17872	23	127720	39015.99	8.25%	52.8/hour	0.01	0.01	410.7
Ask Chat	94	52226	199	251253	61720.74	47.87%	51.2/hour	0.02	0.00	1376.8
TOTAL	399	27004	9	251253	50628.66	21.05%	3.6/min	0.12	0.02	2118.7

Nota: elaboración propia



Nota: elaboración propia

Summary Report

Name: Summary Report

Comments:

Write results to file / Read from file

Filename: Log/Display Only: ☐ Errors ☐ Successes

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
Login	39	13947	3371	62449	11103.59	0.00%	36.9/min	0.38	0.15	629.0
Get Documents	29	8620	4195	43318	7201.81	0.00%	29.1/min	6.61	0.14	13988.9
Upload File	15	15743	11301	47149	8716.62	0.00%	16.2/min	0.07	0.19	274.0
TOTAL	83	12410	3371	62449	9895.58	0.00%	1.3/sec	6.69	0.44	5232.8

Nota: elaboración propia

Para las pruebas de carga y estrés se utilizaría la herramienta JMeter donde se evidenciaron los siguientes resultados

En el escenario 1 se comenzaría a generar usuarios solo para el servicio login mas concretamente 10 usuarios, el ramp-up period de 5 segundos y el loop count de 2

En el escenario 2 se implementaría mas http request que se configuraban para realizar el flujo completo del proyecto login → get documents → upload files → ask chat, se comenzaría con 10 usuarios, el ramp-up period de 5 segundos y el loop count de 2

En el escenario 3 se comenzarían a aumentar las cargas con mas usuarios +50 con la misma configuración anteriormente mencionada

En el escenario 4 ya se implementarían 100 usuarios, el ramp-up period de 10 segundos y el loop count de 2

En el escenario 5 se amplió drásticamente la cantidad de usuarios pasando a 300 soportandolo sin ningún error

Sonar qube cloud



Nota: elaboración propia

Para este proyecto se utilizaría sonar qube cloud implementándose a través de git actions evidenciándose este resultado, métricas de A en todos los parámetros siendo esta la calificación mas alta y con un porcentaje de 0.0% en código duplicado analizando las 5.900 lineas de código

```

$ docker exec venom_backend pytest tests/ -v --cov=app

----- coverage: platform linux, python 3.11.15-final-0 -----
Name                                                                    Stmts   Miss  Cover
-----
app/__init__.py                                                            0      0   100%
app/database.py                                                            12     12     0%
app/services/audit_service.py                                              39      7    82%
app/services/chart_service.py                                             99     52    47%
app/services/dispatchers/__init__.py                                       37      3    92%
app/services/dispatchers/audit_dispatcher.py                             15      2    87%
app/services/dispatchers/base.py                                           15      2    87%
app/services/dispatchers/chart_dispatcher.py                             56     22    61%
app/services/dispatchers/compare_documents_dispatcher.py                 34     10    71%
app/services/dispatchers/email_dispatcher.py                             18      3    83%
app/services/dispatchers/explorer_dispatcher.py                           18      2    89%
app/services/dispatchers/legacy_dispatcher.py                             19      4    79%
app/services/dispatchers/list_documents_dispatcher.py                     21      2    90%
app/services/dispatchers/mcp_apps_dispatcher.py                           18      4    78%
app/services/dispatchers/mcp_music_dispatcher.py                         18      4    78%
app/services/dispatchers/mcp_system_dispatcher.py                        21      7    67%
app/services/dispatchers/mcp_windows_dispatcher.py                       18      4    78%
app/services/dispatchers/pdf_dispatcher.py                                27      2    93%
app/services/dispatchers/process_file_dispatcher.py                       22      6    73%
app/services/dispatchers/web_search_dispatcher.py                         18      4    78%
app/services/document_comparison_service.py                               25      5    80%
app/services/email_service.py                                              72     54    25%
app/services/file_processor.py                                             122     57    53%
app/services/mcp_client.py                                                 91     51    44%
app/services/report_service.py                                             88     43    51%
app/services/vision_service.py                                             69     34    51%
app/services/web_search_service.py                                         52     36    31%
-----
TOTAL                                                                    1044    432    59%

===== short test summary info =====
FAILED tests/test_extra_coverage.py::test_mcp_system_dispatcher - assert ("Er...
FAILED tests/test_more_coverage.py::test_email_send_missing_config - assert F...
===== 2 failed, 38 passed, 2 skipped, 56 warnings in 4.48s =====

```

Nota: elaboración propia

Propuestas de mejora

Mejoras en Seguridad y Aislamiento

Seguridad a nivel de base de datos: Implementar Row-Level Security (RLS) en PostgreSQL como segunda capa de protección contra fugas de datos entre empresas.

Auditoría proactiva: Crear un sistema de alertas en tiempo real ante intentos de acceso no autorizado entre tenants (cross-tenant).

Gestión de secretos: Centralizar credenciales (API Keys, DB) en un gestor especializado (como AWS Secrets Manager) para eliminar archivos .env y asegurar la rotación automática.

Mejoras en Rendimiento

Caché semántico: Almacenar respuestas a consultas frecuentes para reducir latencia, ahorrar consumo de tokens y disminuir costos de la API del LLM.

Procesamiento asíncrono: Migrar la vectorización de documentos a una cola de tareas (Celery/Redis) para evitar bloqueos en el sistema durante la carga de archivos pesados.

Paginación: Implementar paginación en el historial de chat para optimizar el uso de memoria y mantener la velocidad de respuesta conforme el historial crece.

Mejoras en Calidad y Cobertura de Pruebas

Aumento de cobertura: Elevar la cobertura de pruebas unitarias al 80% e implementar mocks robustos para servicios externos.

Seguridad automatizada (DAST): Integrar herramientas como OWASP ZAP en el flujo de despliegue para detectar vulnerabilidades continuamente.

Validación de aislamiento: Crear una suite de pruebas específica que asegure la integridad del multi-tenant ante posibles intentos de acceso indebido.

Pruebas de carga extendidas: Ampliar las pruebas con JMeter para incluir el proceso completo de RAG y carga de documentos, identificando cuellos de botella reales.

Conclusiones

- La arquitectura RAG implementada demuestra ser la estrategia correcta para mitigar el riesgo de alucinaciones en la IA, garantizando que las respuestas del sistema estén siempre ancladas en el conocimiento real de la organización. Esto representa una ventaja diferencial frente a soluciones que utilizan modelos de lenguaje sin contexto documental.
- La implementación de la arquitectura multi-tenant mediante aislamiento por `id_empresa` en todas las entidades críticas sienta las bases para una solución comercialmente escalable. Sin embargo, la evolución hacia Row-Level Security en PostgreSQL es recomendable para elevar el nivel de garantía de seguridad.
- Los resultados de las pruebas de carga con JMeter, particularmente la estabilidad del sistema con 300 usuarios concurrentes sin errores, validan la robustez de la arquitectura FastAPI asíncrona seleccionada. La calificación A en SonarQube con 0% de código duplicado confirma la disciplina de calidad del equipo.
- La metodología Scrum aplicada, con sprints de dos semanas y validaciones incrementales, fue determinante para gestionar la volatilidad inherente al trabajo con modelos de lenguaje. La detección temprana de problemas de integración entre módulos redujo significativamente el riesgo de fallos en etapas avanzadas.

- El proyecto sienta las bases para una plataforma de inteligencia operativa genuinamente útil para el ecosistema de microempresas colombianas, un sector que históricamente carece de acceso a herramientas de gestión del conocimiento de este nivel de sofisticación.

<https://venom->

project.atlassian.net/jira/software/projects/SCRUM/summary?atlOrigin=eyJpIjoiZjJlNTU3YzcyM2EwNDRhNGJmNjA2ODkxODQ1ZWNI

Anexos

- Jira <https://venom-project.atlassian.net/jira/software/projects/SCRUM/summary?atlOrigin=eyJpIjoiZjJlNTU3YzcyM2EwNDRhNGJmNjA2ODkxODQ1ZWNlOGEiLCJwIjoiajJ9>

Referencias

Anthropic. (2024). Claude API Documentation. Recuperado de <https://docs.anthropic.com>

Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., ... &

Thomas, D. (2001). Manifesto for Agile Software Development. Agile Alliance.

Recuperado de

<https://agilemanifesto.org>

Chroma. (2024). Chroma: The AI-native open-source embedding database. Recuperado de <https://docs.trychroma.com/>

<https://www.trychroma.com>

Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley Professional.

Gartner. (2024). Magic Quadrant for Cloud Database Management Systems. Gartner Research.

- Grimaldi, A., & Mago, V. K. (2023). Multi-Tenant Architecture Patterns for SaaS Applications. *Journal of Cloud Computing*, 12(1), 45–62.
- LangChain. (2024). LangChain: Build context-aware reasoning applications. Recuperado de <https://python.langchain.com>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 9459–9474.
- Martin, R. C. (2003). *Agile Software Development, Principles, Patterns, and Practices*. Prentice Hall.
- OpenAI. (2024). OpenAI API Documentation. Recuperado de <https://platform.openai.com/docs>
- PostgreSQL Global Development Group. (2024). PostgreSQL 16 Documentation. Recuperado de <https://www.postgresql.org/docs/16/>
- Ramírez, J., & Torres, C. (2022). Gestión del Conocimiento en Microempresas: Retos y Oportunidades en la Transformación Digital. *Revista Iberoamericana de Sistemas, Cibernética e Informática*, 19(2), 12–21.
- Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide: The Definitive Guide to Scrum — The Rules of the Game*. Scrum.org.
- Wieringa, R. J. (2014). *Design Science Methodology for Information Systems and Software Engineering*. Springer.
- Wiggins, A. (2017). The Twelve-Factor App. Recuperado de <https://12factor.net>